Philipps-University Marburg
Department Mathematics and Computer Science
Working Group Database Systems

Wintersemester 2025
21.11.2025

Prof. Dr. Bernhard Seeger
Manuel Oed

Exercises for the lecture
NoSQL-Database Systems

Submission: 04. 12. 2025,
no later than 16:00 Uhr
via ILIAS

Exercise 3

Task 3.1: Map-Reduce & Java Streams (3+4+4+4) (15 Points)

In the following, we consider a few simple tasks that we want to express in a Map-Reduce framework. Except for `MAP2` in Task **d)**, the input for the Map functions that we consider will be a document-ID (*number*) and the document content ($text$). The `split`-Method can be used to split a text into an array of its words. For a string `w`, the i-th character can be accessed via `w.get(i)`, and `w.length()` returns the length of `w` (i.e., the number of characters).

a) Consider the following definitions of the map and the reduce functions.

MAP($number, text$)
  words ← split(' ', $text$)
  for $i \leftarrow 0, \ldots,$|words|-1 **do**
    emit(words[i], 1)

REDUCE($key, values$)
  $a \leftarrow 0$
  for $v : values$ **do**
    $a \leftarrow a + v$
  emit(key, a)

(i) With these definitions, what does the Map-Reduce framework compute for a given set of documents?

Consider the two documents given as `<number, text>`:
`<1, ''Lorem ipsum Hello ipsum dolor''>` and
`<2, ''Hello Hello world''>`

(ii) State all output tuples produced by the map function when passing the two documents.

(iii) State all input tuples passed to the reduce function by the Map-Reduce framework.

b) Define Map and Reduce functions to compute all characters that occur as the first letter of at least 100 words across all documents. Use the same input/output signature and emit pattern as in Task **a)**. The reduce function should emit pairs consisting of a qualifying character and the number of its occurrences as the first letter in any word.

c) Define Map and Reduce functions to compute the average word-length per document, using the same input/output signature and emit pattern as in Task **a)**. The reduce function should emit pairs consisting of a document-ID and the corresponding average word-length.

d) Some tasks can not be solved immediately within one round of Map and Reduce. However, the emitted pairs of a reduce function can again be used as the input for a second round of Map and reduce.
Define the functions `MAP1`, `REDUCE1`, `MAP2`, and `REDUCE2` to compute the maximum number

of times any single word appears across all incoming documents. For example, considering the two documents provided in Task **a)** as input, the result would be 3. The resulting tuple emitted by `REDUCE2` should return the required number as value; the associated key does not matter.

Task 3.2: Map-Reduce and Java Streams (3+3+3+3+4+4) (20 Points)

The MapReduce programming model is inspired by the map and reduce functions often used in functional programming languages and is frequently used for analyzing log files. Accordingly, the basic programming concepts used in MapReduce frameworks such as Apache Hadoop can also be practiced using functional programming languages. In this task, you will use the Java Streams API to analyze log files.

To do this, work on the class `MovieShopAnalysis` in the provided project template. In the template, you will also find a class `MovieShopLogEntry`, which models a purchase in a movie store. Additionally, the template includes a file `movieshoplog`, which contains $458,428$ serialized `MovieShopLogEntry` objects. You can load these into a list in main memory using the provided `MovieShopLogReader` class.

Start each solution by treating the list as a stream (`.stream()`). Then execute a `.map()` command as the first operation on the stream, which merges the objects into an `AbstractMap.SimpleEntry<>`. This roughly corresponds to the output of a Map process, which produces (key, value) pairs. Starting from this point, use only the Java Streams API for further processing — i.e., avoid loops and similar constructs.

After completing the tasks, test your solution in a main method by loading the movieshoplog file and running your completed methods on it.

a) Complete the method `ordersPerCustomer` so that it outputs the total number of movies purchased by each customer.

b) Complete the method `nMoviesPerCategory`, which computes the number of movies in each category.

c) Complete the method `topFiveCategories` so that it outputs the five movie categories with the highest number of films sold.

d) Complete the method `fifteenMoviesPerDay` by identifying all customers who purchased at least 15 movies on a single day, and output these customers together with the number of movies and the corresponding day.

e) Complete the method `topCustomer`, which calculates the customer who spent, among all customers, the most money on movies.

f) Complete the method `allCategories`, which calculates all customers who bought at least one movie from each category.

Hint: Use the `Stream.collect()` method combined with Collectors.